
Description of project tasks: version Zwei

TANA21/22: BERÄKNINGSMATEMATIK

Andrew R. Winters

HT1 2026

The problem to be solved in the project will be the numerical solution of a two-point boundary value problem governed by a second-order, linear ordinary differential equation (ODE)

$$\begin{aligned}y''(x) &= p(x)y'(x) + q(x)y(x) + r(x) \quad \text{where } x \in (a, b), \\y(a) &= \alpha, \\y(b) &= \beta.\end{aligned}$$

Physical applications of such a two-point boundary value problem are numerous such as the load and deformation of a length of a beam or the heating of a wire as electricity is run through it. This problem uses an ODE model for a function in the interior of the interval where the function is fixed with *boundary conditions* by two constant values at either side of the interval.

Below we divide the end goal to solve this two-point boundary value problem into two milestones. This structure will help organize the numerical approximation into manageable pieces that will also assist in debugging the code. In a sense, the first milestones acts as your computational “toolbox” that will be applied to solve the larger problem. Broadly, the milestones are implementing and testing:

1. A routine to solve a linear system $\mathbf{Ax} = \mathbf{b}$ where $\mathbf{A}$ is tridiagonal.
2. A routine to solve the two-point boundary value problem given above.

1. SOLVER FOR A TRIDIAGONAL SYSTEM OF EQUATIONS

Write a linear solver routine for a linear system $\mathbf{Ax} = \mathbf{b}$ where the matrix $\mathbf{A}$ is tridiagonal. The implementation should be done in python.

To solve the tridiagonal system is a computationally efficient way you will use the *Thomas algorithm*. This is a special flavour of the **LU** factorization that exploits the tridiagonal nature of the matrix $\mathbf{A}$. Information and details on the Thomas algorithm can be found in the textbook (pg. 219-220) or through many online resources¹.

Below are two tasks that should be performed for this milestone:

- Verify that the implementation of the Thomas algorithm correctly solves the linear system $\mathbf{Ax} = \mathbf{b}$. You can use the commands below to generate non-singular tridiagonal square matrices of size N for which the Thomas algorithm is guaranteed to work:

```
# create three random vectors
main = N * np.ones((N,)) + np.random.rand((N,))
sub = np.random.rand((N - 1,))
super = np.random.rand((N - 1,))
# use the 'np.diag' command to create tridiagonal matrix
A = np.diag(sub, k=-1) + np.diag(main, k=0) + np.diag(super, k=1)
```

Once the matrix $\mathbf{A}$ is created you can “manufacture” a right-hand-side vector $\mathbf{b}$ for which you know the solution $\mathbf{x}$ a priori. For instance, to create a problem where you know that the solution $\mathbf{x}$ is a vector of all ones you can use the commands:

¹https://sv.wikipedia.org/wiki/Tridiagonal_matris

```
known_x = np.pi * np.ones((N,)) # create known solution vector filled with pi
manuf_b = A @ known_x           # manufacture the right-hand-side
```

- Perform a cost analysis for the tridiagonal solver via the Thomas algorithm. Demonstrate that the computational cost is $\mathcal{O}(N)$. This cost analysis can be done with CPU timings for varying matrix sizes and/or via a flop counting argumentation. How does this compare to the computational cost of the standard **LU** factorization using Gaussian elimination with pivoting?

2. SOLVING A TWO-POINT BOUNDARY VALUE PROBLEM

Use first and second central difference approximations for the derivatives in the following ODE

$$y'' = -\frac{4}{x}y' - \frac{2}{x^2}y + \frac{2\ln(x)}{x^2}, \quad 1 \leq x \leq 2 \quad \text{with} \quad y(1) = \frac{1}{2} \quad \text{and} \quad y(2) = \ln(2)$$

The goal of this task is to numerically approximate the solution of the above ODE on the given interval with the prescribed boundary conditions. This ODE has the exact solution

$$y = \frac{4}{x} - \frac{2}{x^2} + \ln(x) - \frac{3}{2}.$$

This exact solution should be used to verify the solution accuracy of the approximate solution.

Below are the tasks that should be performed for this milestone:

- Discuss the discretization of the ODE using central finite difference approximations. Show how to assemble the discrete problem into a matrix vector representation of the form $\mathbf{A}\mathbf{y} = \mathbf{b}$. (**OBS!** Exercise 5.13 from Exercise Set 5 can guide this construction and discussion).
- Solve for the unknown values of $\mathbf{y}$ in the resulting tridiagonal linear system using your implementation of the Thomas algorithm from the first milestone.
- Compute the errors in the vector ∞ -norm between your approximate solution and the exact solution on an increasing set of nodes $n = \{10, 20, 40, 80\}$. Verify that your approximation is second order accurate (**Hint:** look at ratios between the two subsequent error values).